

Snapper Chain (Hard)

Day 1 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“Snapper devices and chained them together”*
- ▶ *“Will the light be on or off”*

Programming interpretation:

- ▶ The chain state follows the low bits of a binary counter.
- ▶ The light is on exactly when all N snappers are on and powered.
- ▶ That condition is equivalent to the lowest N bits of K all being 1.

Textbook connection: CP4 2.2 h / 2.2.3 Bitmask

Core technique: Bitmask pattern recognition

- ▶ The chain behaves like a binary counter after each clap.
- ▶ The light is on only when all N snappers are on.
- ▶ That means the lowest N bits of K must all be 1.

- ▶ Simulating K claps is unnecessary and may be too slow.
- ▶ The repeated states reveal powers of two.
- ▶ A mask with N low bits set tests the condition in constant time.

- ▶ For each case, build $\text{mask} = (1 \ll N) - 1$.
- ▶ Compute $K \& \text{mask}$ to isolate the lowest N bits.
- ▶ If the isolated bits equal mask, print ON.
- ▶ Otherwise print OFF.

- ▶ For $N = 3$, mask is binary 111.
- ▶ $K \& 111$ discards all higher bits of K .
- ▶ Only values ending in 111 have all three snappers on.

- ▶ The snapper chain advances through the same states as binary counting.
- ▶ A snapper is powered only if every lower snapper is on, so the final light depends on all N low bits.
- ▶ The bitmask check is exactly the condition that all those bits are 1.

Complexity: $O(T)$ time and $O(1)$ extra memory for T test cases.

- ▶ Parentheses around $1 \ll N$ make the mask expression clear.
- ▶ This is a good example of replacing simulation with a state-pattern observation.

Source lines: `snapperhard.py:39-44`

```
import sys

data = sys.stdin.read().strip().splitlines()

for idx, case in enumerate(data[1:]):
    n, k = list(map(int, case.split()))
```

What this chunk does

- ▶ The first input line is the number of cases.
- ▶ Each case supplies N, the chain length, and K, the number of snaps.
- ▶ `enumerate` gives the 1-based case labels used in the output.

Source lines: `snapperhard.py:45-49`

```
mask = (1 << n) - 1 # check if all lower bits below n are turned on
if (k & mask) == mask: # all n low bits are 1
    print(f"Case #{idx+1}: ON")
else:
    print(f"Case #{idx+1}: OFF")
```

What this chunk does

- ▶ The mask contains N ones in binary.
- ▶ `k & mask` isolates exactly the N snappers that matter.
- ▶ If those bits equal the mask, every snapper in the chain is ON.

Solution file: `practice_contests/day_1/snapperhard.py`

Problem: <https://open.kattis.com/problems/snapperhard>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.